

Para el ecosistema de Flutter, el estándar absoluto de la industria es el paquete oficial **local_auth**, desarrollado directamente por el equipo de Google. Este paquete no inventa una biometría propia; actúa como un puente unificado hacia las APIs nativas de máxima seguridad de los sistemas operativos:

BiometricPrompt en Android y **LocalAuthentication** (Touch ID / Face ID) en iOS.

Seguridad Móvil y Autenticación Biométrica Hardware

Este módulo práctico enseña a integrar mecanismos de seguridad criptográfica local mediante el uso de los sensores biométricos del dispositivo (huella digital o rostro), gestionando los estados de hardware disponibles y abstrayendo la lógica nativa del sistema operativo de forma reactiva.

1. Fundamentos Teóricos y Conceptos del Paquete **local_auth**

En el desarrollo móvil profesional, la biometría está fuertemente protegida por zonas seguras de hardware en el procesador (como el *Trusted Execution Environment* en Android o el *Secure Enclave* en Apple).

- **Aislamiento Biométrico (Sandbox):** Un concepto crítico que deben entender es que **nuestra aplicación nunca tiene acceso a la foto del rostro del usuario ni a la imagen de su huella dactilar**. El hardware realiza la validación de forma aislada y solo le responde a Flutter un token booleano: **true** (autenticado) o **false** (rechazado).
 - **BiometricType (Tipos de Hardware):** Es el enumerador que clasifica qué sensores físicos tiene el teléfono:
 - **BiometricType.fingerprint:** Sensor físico o ultrasónico de huellas dactilares.
 - **BiometricType.face:** Reconocimiento facial por infrarrojos o cámara.
 - **BiometricType.weak / strong:** Clasificación moderna de Android basada en el nivel de seguridad y resistencia a suplantaciones (criptográfica).
 - **Mecanismos de Respaldo (Fallback):** Si el dispositivo no tiene biometría configurada o el sensor falla, el paquete permite degradar la seguridad de forma elegante hacia el código PIN, patrón de puntos o contraseña del sistema operativo que el usuario configuró originalmente en su pantalla de bloqueo.
-

2. Configuración del Entorno (Plataforma Nativa)

La autenticación biométrica requiere interactuar con el ciclo de vida de las actividades nativas de Android y declarar las llaves de privacidad en iOS.

En Android

1. **Modificar la Actividad Principal (**android/app/src/main/abs.../MainActivity.kt** o **.java**):** Para que los diálogos biométricos se pinten correctamente por encima de Flutter, la app debe heredar de **FlutterFragmentActivity** en lugar de **FlutterActivity**. Abre tu archivo **MainActivity.kt** y modifícalo para que quede así:

```
import io.flutter.embedding.android.FlutterFragmentActivity

class MainActivity: FlutterFragmentActivity() {
}
```

2. **Declarar el Permiso** (**android/app/src/main/AndroidManifest.xml**): Añade la siguiente línea de hardware justo antes de la etiqueta `<application>`:

```
<uses-permission android:name="android.permission.USE_BIOMETRIC" />
```

En iOS (**ios/Runner/Info.plist**)

Añade la llave obligatoria para el uso de Face ID (Touch ID no requiere llave explícita, pero Face ID sí):

```
<key>NSFaceIDUsageDescription</key>
<string>Necesitamos acceso al sensor facial para validar tu identidad en la
práctica de Biometría.</string>
```

Estructura de Carpetas del Módulo

Añadimos la nueva característica (**feature**) llamada **biometrics** dentro de la arquitectura limpia de nuestro proyecto:

```
mi_app_sensores/
├── pubspec.yaml
├── lib/
│   ├── main.dart
│   ├── core/
│   │   └── theme/
│   │       └── theme.dart
│   └── features/
│       ├── home/
│       └── biometrics/          <-- CARPETA DEL MÓDULO ACTUAL
│           └── screens/
│               └── biometric_screen.dart
```

Agreguen la dependencia oficial en su **pubspec.yaml**:

```
dependencies:  
  flutter:  
    sdk: flutter  
  local_auth: ^2.3.0 # O la versión estable más reciente
```

3. Código Fuente Completo y Comentado

Archivo `lib/features/biometrics/screens/biometric_screen.dart`

```
import 'package:flutter/material.dart';  
import 'package:local_auth/local_auth.dart';  
import 'package:flutter/services.dart';  
  
class BiometricScreen extends StatefulWidget {  
  const BiometricScreen({super.key});  
  
  @override  
  State<BiometricScreen> createState() => _BiometricScreenState();  
}  
  
class _BiometricScreenState extends State<BiometricScreen> {  
  // Controlador central de la API de Autenticación Local  
  final LocalAuthentication _auth = LocalAuthentication();  
  
  String _hardwareStatus = "Verificando sensores...";  
  String _authStatus = "Estado: Esperando acción";  
  bool _canCheckBiometrics = false;  
  List<BiometricType> _availableBiometrics = [];  
  
  @override  
  void initState() {  
    super.initState();  
    _checkHardwareCapabilities();  
  }  
  
  // 1. Auditoría de Hardware: ¿Qué sensores tiene físicamente este teléfono?  
  Future<void> _checkHardwareCapabilities() async {  
    bool canCheckBiometrics = false;  
    bool isDeviceSupported = false;  
    List<BiometricType> availableBiometrics = [];  
  
    try {  
      // Evalúa si el chip del teléfono soporta biometría o tiene bloqueo de  
      // pantalla activo  
      isDeviceSupported = await _auth.isDeviceSupported();  
      // Evalúa si el sensor físico está disponible para usarse en este momento  
      canCheckBiometrics = await _auth.canCheckBiometrics;
```

```

        if (canCheckBiometrics) {
            // Extrae la lista de tecnologías biométricas enroladas en el sistema
operativo
            availableBiometrics = await _auth.getAvailableBiometrics();
        }
    } on PlatformException catch (e) {
        debugPrint("Error de plataforma al auditar hardware: $e");
    }

    if (!mounted) return;

    setState(() {
        _canCheckBiometrics = canCheckBiometrics && isDeviceSupported;
        _availableBiometrics = availableBiometrics;

        _hardwareStatus = _canCheckBiometrics
            ? "Sensores Disponibles: ${_parseBiometricsList()}"
            : "❌ Este dispositivo no cuenta con biometría activa.";
    });
}

String _parseBiometricsList() {
    if (_availableBiometrics.isEmpty) return "Ninguno (Usa PIN/Patrón de
respaldo)";
    return _availableBiometrics.map((e) =>
e.toString().split('.').last.toUpperCase()).join(", ");
}

// 2. Disparar el flujo asíncrono de Autenticación Criptográfica
Future<void> _authenticateUser() async {
    if (!_canCheckBiometrics) {
        setState(() { _authStatus = "Acción cancelada: Hardware no disponible."; });
        return;
    }

    setState(() { _authStatus = "Estado: Autenticando..."; });

    try {
        // Lanzamos la interfaz nativa del Sistema Operativo (BiometricPrompt /
FaceID)
        bool authenticated = await _auth.authenticate(
            localizedReason: 'Escanea tu huella o rostro para validar el laboratorio',
        );

        setState(() {
            _authStatus = authenticated
                ? "🔒 ACCESO CONCEDIDO - Identidad Verificada"
                : "❌ ACCESO DENEGADO - Intento Fallido";
        });
    } on PlatformException catch (e) {
        setState(() {
            _authStatus = "Error de autenticación: ${e.message}";
        });
    }
}

```

```

}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Sensor: Autenticación Biométrica')),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          // Panel 1: Estado del Hardware del Dispositivo
          Card(
            elevation: 4,
            child: Padding(
              padding: const EdgeInsets.all(16.0),
              child: Column(
                children: [
                  const Text('Auditoría de Capacidades Físicas', style:
TextStyle(fontSize: 16, fontWeight: FontWeight.bold)),
                  const SizedBox(height: 12),
                  Text(_hardwareStatus, textAlign: TextAlign.center, style:
const TextStyle(fontFamily: 'monospace', color: Colors.blueGrey)),
                ],
              ),
            ),
          ),
          const SizedBox(height: 25),

          // Panel 2: Consola de Control de Seguridad
          Card(
            elevation: 4,
            color: _authStatus.contains("CONCEDIDO") ?
Colors.green.withValues(alpha:0.05) : Colors.red.withValues(alpha:0.05),
            child: Padding(
              padding: const EdgeInsets.all(24.0),
              child: Column(
                children: [
                  Icon(
                    _authStatus.contains("CONCEDIDO") ? Icons.lock_open :
Icons.lock,
                    size: 64,
                    color: _authStatus.contains("CONCEDIDO") ? Colors.green :
Colors.redAccent,
                  ),
                  const SizedBox(height: 16),
                  Text(
                    _authStatus,
                    textAlign: TextAlign.center,
                    style: const TextStyle(fontSize: 15, fontWeight:
FontWeight.bold),
                  ),
                ],
              ),
            ),
          ),
        ],
      ),
    ),
  );
}

```

```

    ),
  ),
),
const SizedBox(height: 35),

// Botón de acción central
ElevatedButton.icon(
  onPressed: _canCheckBiometrics ? _authenticateUser : null,
  style: ElevatedButton.styleFrom(
    padding: const EdgeInsets.symmetric(vertical: 16),
    backgroundColor: Colors.indigo,
    foregroundColor: Colors.white,
  ),
  icon: const Icon(Icons.fingerprint),
  label: const Text('Disparar Validación Biométrica', style:
TextStyle(fontSize: 16)),
),
],
),
),
);
}
}

```

4. Guía Didáctica

- **El Rol de `FlutterFragmentActivity`:** La clase estándar `FlutterActivity` dibuja una vista plana de pantalla completa. Como los diálogos de huella digital son fragmentos flotantes nativos del sistema operativo, cambiar la herencia de la app a `FlutterFragmentActivity` en Android es obligatorio para habilitar el soporte de ventanas superpuestas multitarea.
- **La Bandera `stickyAuth`:** Si el usuario está poniendo su huella y accidentalmente le entra una llamada telefónica o el sensor apaga la pantalla, `stickyAuth` evita que el hilo de Flutter muera con un error. En su lugar, pausa la ejecución y reanuda el escaneo en cuanto la app vuelve a primer plano.

Vinculación en el Menú Principal

Abre el archivo maestro `lib/features/home/screens/home_screen.dart` para activar el acceso al nuevo laboratorio:

```

// 1. Agrega el import correspondiente al inicio de home_screen.dart:
import '../biometrics/screens/biometric_screen.dart';

// 2. Agrega la tarjeta correspondiente al GridView:
_SensorCard(
  title: 'Biometría (Huella/Rostro)',
  icon: Icons.fingerprint,
  color: Colors.indigo, // Color índigo representativo de seguridad

```

```
onTap: () {  
  Navigator.push(  
    context,  
    MaterialPageRoute(builder: (context) => const BiometricScreen()),  
  );  
},  
,
```